

BigQuery Query Cheat Sheet

This cheat sheet provides BigQuery (GoogleSQL) queries to assist during the training labs.

Contact us
training@invictus-ir.com

Your tables

All tables live in project **training-496707**. Reference them as ``project.dataset.table`` in backticks.

Cloud audit logs (Lab 1.x)

lab1_2_logs.audit_log_v1
lab1_3_logs.audit_log_v1
lab1_4_logs.audit_log_v1
lab1_4_logs.data_access_log_v1

Workspace logs (Lab 2.x)

lab2_3_gws / lab2_4_gws / lab2_5_gws → login_events,
gmail_log_events, drive_log_events, token_log_events

lab2_5 also: login_events_enriched, context_aware_access

> Querying Basics

Querying a table

Specify the fully-qualified table in backticks. On its own it returns every row and column.

```
SELECT *
FROM `training-496707.lab1_2_logs.audit_log_v1`
```

Names are project.dataset.table. Use the Schema and Preview tabs in the editor to see fields first.

Limit the rows

Cap the rows returned while exploring so queries stay fast and cheap.

```
SELECT *
FROM `training-496707.lab1_2_logs.audit_log_v1`
LIMIT 10
```

BigQuery still scans the columns you select, so select only what you need.

Select specific columns

List only the fields you need instead of `*`.

```
SELECT timestamp,
  protopayload_auditlog.methodName
FROM `training-496707.lab1_2_logs.audit_log_v1`
```

Nested fields use dot notation (see Workspace & nested data).

Unique values

Return each distinct value once - handy for listing IPs, emails or methods.

```
SELECT DISTINCT
  protopayload_auditlog.requestMetadata.callerIp
FROM `training-496707.lab1_3_logs.audit_log_v1`
```

DISTINCT collapses duplicates across all selected columns.

> Filtering Data

Where clause

Keep only rows that match a condition. String values go in single quotes.

```
SELECT timestamp, protopayload_auditlog.methodName
FROM `training-496707.lab1_2_logs.audit_log_v1`
WHERE protopayload_auditlog
  .authenticationInfo.principalEmail =
  'compromised-dev@invictus-training.com'
```

Combine conditions with AND / OR.

Partial match with LIKE

Match part of a string using % as a wildcard.

```
WHERE protopayload_auditlog.methodName
LIKE 'v1.compute.instances%'
```

`%text%` matches anywhere, `'text%'` matches the start. Use NOT LIKE to exclude, e.g. callerIp NOT LIKE '10.%' to drop internal IPs.

Match several values (IN)

Match any value from a list - shorter than many OR conditions.

```
WHERE protopayload_auditlog.methodName IN (
  'jobservice.jobcompleted',
  'google.cloud.bigquery.v2.JobService.InsertJob')
```

Returns rows where the column equals any listed value.

Earliest / latest event

Sort by time and take the top row to find the first (or last) occurrence.

```
SELECT timestamp,
  protopayload_auditlog.resourceName
FROM `training-496707.lab1_3_logs.audit_log_v1`
WHERE protopayload_auditlog.methodName
= 'SetIamPolicy'
ORDER BY timestamp ASC
LIMIT 1
```

ASC = earliest, DESC = latest. timestamp is a top-level column.

> Counting & Summarizing

Count rows

Total the rows that match your filters.

```
SELECT COUNT(*) AS denials
FROM
  `training-496707.lab2_5_gws.context_aware_access`
```

Answers that want 'digits only' usually need COUNT.

Count unique values

Count distinct values rather than rows.

```
SELECT COUNT(DISTINCT actor.email) AS victims
FROM `training-496707.lab2_3_gws.login_events`
WHERE ipAddress = '45.155.205.18'
```

AS names the output column.

Group & rank (most frequent)

Group rows and count each group, then sort to find the top value.

```
SELECT
  protopayload_auditlog.requestMetadata.callerIp,
  COUNT(*) AS n
FROM `training-496707.lab1_3_logs.audit_log_v1`
GROUP BY 1
ORDER BY n DESC
```

GROUP BY 1 groups by the first selected column; DESC puts the noisiest on top.

First-seen, totals & sizes

Aggregate with MIN / MAX / SUM. Combine SUM with arithmetic to convert units.

```
SELECT ROUND(
  SUM(protopayload_auditlog.response.size)
  / 1024 / 1024) AS mb
FROM `training-496707.lab1_4_logs.audit_log_v1`
WHERE protopayload_auditlog.methodName
= 'storage.objects.create'
```

MIN(timestamp) gives first-seen; /1024/1024 converts bytes to MB.

> Workspace & Nested Data

Nested fields (dot notation)

Audit-log fields are nested STRUCTs - reach them with dots. No JSON parsing needed.

```
SELECT DISTINCT protopayload_auditlog
  .requestMetadata.callerSuppliedUserAgent
FROM `training-496707.lab1_2_logs.audit_log_v1`
```

e.g. ...authenticationInfo.principalEmail, ...requestMetadata.callerIp.

Flatten Workspace events (UNNEST)

Workspace logs wrap activity in a repeated events array with a name/value parameter bag. UNNEST to flatten them.

```
SELECT COUNT(DISTINCT actor.email) AS clickers
FROM `training-496707.lab2_3_gws.gmail_log_events`,
  UNNEST(events) e, UNNEST(e.parameters) p
WHERE e.name = 'email_link_click'
  AND p.name = 'url'
  AND p.value LIKE '%invictus-trainings.com%'
```

Filter events on e.name, then match a parameter with p.name / p.value. Workspace uses actor.email, ipAddress and id.time.

Pull a parameter into a column

Use a subquery in the SELECT list to turn one parameter into a column.

```
SELECT actor.email, id.time,
  (SELECT p.value FROM UNNEST(e.parameters) p
   WHERE p.name = 'doc_title') AS doc_title
FROM `training-496707.lab2_4_gws.drive_log_events`,
  UNNEST(events) e
WHERE e.name = 'download'
ORDER BY id.time ASC
LIMIT 1
```

Numeric parameters live in p.intValue (e.g. num_bytes), not p.value.

Scopes & time between events

Multi-value parameters (OAuth scopes) need one more UNNEST. Use TIMESTAMP_DIFF for durations.

```
-- one scope per row
UNNEST(e.parameters) sp,
UNNEST(sp.multiValue) scope
WHERE sp.name = 'scope'

-- dwell time in seconds
TIMESTAMP_DIFF(last_seen, first_seen, SECOND)
```

The extra UNNEST exposes each scope value individually.

